EDUCATION INNOVATION CAFE
(EIC)

Nom complet : Françoise NOEL

Titre du devoir : Voltmètre numérique avec écran LCD

Cours : IA & Robotique

Professeur : Carlos Yfrazin

Date : 09 octobre 2026

Sommaire

1. Introduction et objectifs
2. Principes théoriques
3. Simulation 1 : le voltmètre numérique
4. Simulation 2 : la version améliorée
5. Synthèse : rôle de chaque composant
6. Conclusion

1. Introduction et objectifs

Ce devoir consiste à concevoir, simuler et analyser un voltmètre numérique construit autour d'une carte Arduino Uno. Le travail est réalisé en deux étapes, entièrement en simulation sur la plateforme en ligne Wokwi :

- **Simulation 1 :** un voltmètre numérique de base qui affiche la tension mesurée sur un écran LCD 16x2.
- **Simulation 2 :** une version améliorée qui ajoute trois LED (verte, jaune, rouge), un buzzer d'alerte et un bouton HOLD permettant de figer la mesure affichée.

Dans les deux simulations, un potentiomètre représente la tension inconnue à mesurer : en tournant son curseur, on fait varier une tension comprise entre 0 V et 5 V, comme si l'on touchait avec des sondes différents points d'un circuit.

Les objectifs pédagogiques sont les suivants :

- comprendre comment une grandeur analogique (une tension) est convertie en valeur numérique par l'Arduino ;
- savoir afficher des informations sur un écran LCD à l'aide du bus I2C ;
- commander des sorties (LED, buzzer) en fonction d'une mesure, à l'aide de seuils ;
- gérer une entrée numérique (bouton poussoir) avec résistance de rappel interne et anti-rebond ;
- expliquer le rôle de chaque composant dans le montage.

2. Principes théoriques

2.1 Le potentiomètre : un diviseur de tension réglable

Un potentiomètre est une résistance variable à trois broches. Les deux broches extrêmes (VCC et GND) sont reliées aux bornes d'une piste résistive ; la broche centrale (SIG) est reliée à un curseur mobile qui se déplace sur cette piste. Le curseur sépare la résistance totale en deux parties R1 et R2.

Lorsque le curseur est tourné complètement vers GND, la tension de sortie vaut 0 V. Lorsqu'il est tourné complètement vers VCC, elle vaut 5 V. Entre les deux, la tension varie de façon progressive. C'est exactement ce qu'il faut pour simuler une tension inconnue comprise entre 0 et 5 V.

2.2 Le convertisseur analogique-numérique (CAN)

Le microcontrôleur de l'Arduino Uno (ATmega328P) ne sait traiter que des nombres. Pour lire une tension, il utilise un convertisseur analogique-numérique de 10 bits. Avec 10 bits, il peut représenter 2^10 = 1024 valeurs différentes, numérotées de 0 à 1023. La plage de mesure (0 V à 5 V, qui est la tension de référence) est répartie sur ces 1024 valeurs.

La fonction analogRead(A0) renvoie donc un entier entre 0 et 1023. Pour retrouver la tension en volts, on applique la formule :

Tension (V) = valeur × 5,0 / 1023

La résolution du voltmètre, c'est-à-dire la plus petite variation de tension détectable, est de 5 / 1023 ≈ 4,89 mV. Le tableau suivant donne quelques exemples :

Valeur lue (analogRead)	Calcul	Tension affichée
0	0 × 5 / 1023	0,00 V
256	256 × 5 / 1023	1,25 V
512	512 × 5 / 1023	2,50 V
767	767 × 5 / 1023	3,75 V
1023	1023 × 5 / 1023	5,00 V

2.3 Le bus I2C et l'écran LCD

Un écran LCD 16x2 classique demande de nombreuses broches de l'Arduino (environ six). En utilisant un module I2C, la communication se fait avec seulement deux fils : SDA (données) et SCL (horloge). Sur l'Arduino Uno, ces deux lignes se trouvent sur les broches A4 (SDA) et A5 (SCL). Chaque appareil du bus possède une adresse ; celle de l'écran simulé est 0x27. Cela libère les autres broches pour les LED, le buzzer et le bouton.

2.4 Limites du montage

Une entrée analogique de l'Arduino Uno ne supporte pas plus de 5 V. Ce voltmètre est donc limité à la plage 0 à 5 V. Pour mesurer des tensions plus élevées dans un montage réel, il faudrait ajouter un pont diviseur de tension (deux résistances) afin de ramener la tension à mesurer dans la plage admissible, puis corriger la formule en conséquence. Dans ce devoir, le potentiomètre alimenté en 5 V garantit que la tension reste toujours dans la plage sûre.

3. Simulation 1 : le voltmètre numérique

3.1 Composants utilisés

Composant	Qté	Rôle dans le montage
Arduino Uno	1	Cerveau du montage : lit la tension, effectue le calcul et commande l'écran.
Potentiomètre	1	Simule la tension à mesurer (0 à 5 V) grâce à son curseur.
Écran LCD 16x2 (I2C)	1	Affiche la valeur de la tension en volts sur deux lignes de 16 caractères.
Fils de connexion	—	Relient l'alimentation, la masse et les signaux.

3.2 Schéma de câblage

Composant	Broche du composant	Relié à (Arduino)
Potentiomètre	VCC	5V
Potentiomètre	SIG	A0
Potentiomètre	GND	GND
LCD I2C	GND	GND
LCD I2C	VCC	5V
LCD I2C	SDA	A4
LCD I2C	SCL	A5

3.3 Principe de fonctionnement

Le fonctionnement suit un cycle qui se répète en continu, cinq fois par seconde :

- **Étape 1 – Lecture :** l'Arduino lit la tension présente sur la broche A0 (sortie du potentiomètre) et obtient un nombre entre 0 et 1023.
- **Étape 2 – Conversion :** ce nombre est transformé en volts avec la formule V = valeur × 5 / 1023.
- **Étape 3 – Affichage :** la tension est écrite sur la deuxième ligne de l'écran, avec deux décimales, tandis que la première ligne affiche « Tension mesuree ».
- **Étape 4 – Pause :** une courte attente de 200 ms évite que l'affichage ne clignote et laisse l'écran lisible.

3.4 Programme

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Voltmetre num.");
  delay(1500);
  lcd.clear();
}

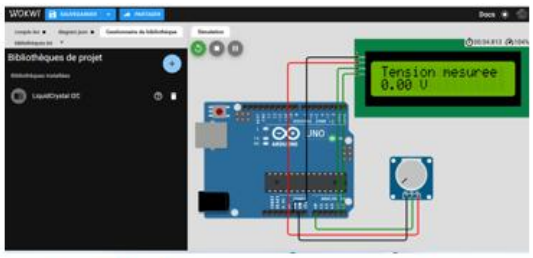
void loop() {
  int valeur = analogRead(A0);              // 0 a 1023
  float tension = valeur * 5.0 / 1023.0;    // conversion en volts

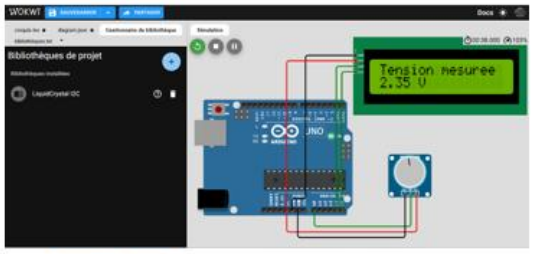
  lcd.setCursor(0, 0);
  lcd.print("Tension mesuree");
  lcd.setCursor(0, 1);
  lcd.print(tension, 2);
  lcd.print(" V    ");
  delay(200);
}
```

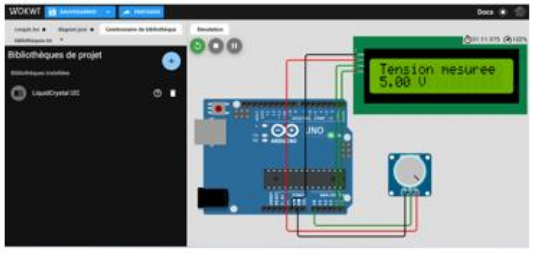
3.5 Explication détaillée du programme

- **#include <Wire.h> et #include <LiquidCrystal_I2C.h> :** chargent la bibliothèque du bus I2C et celle qui pilote l'écran LCD à travers ce bus.
- **LiquidCrystal_I2C lcd(0x27, 16, 2) :** crée l'objet « lcd » en indiquant l'adresse I2C de l'écran (0x27), le nombre de colonnes (16) et de lignes (2).
- **setup() :** cette fonction s'exécute une seule fois au démarrage. lcd.init() initialise l'écran, lcd.backlight() allume le rétroéclairage, puis un message d'accueil « Voltmetre num. » est affiché pendant 1,5 seconde avant d'effacer l'écran avec lcd.clear().
- **loop() :** cette fonction se répète indéfiniment.
- **analogRead(A0) :** lit l'entrée analogique A0 et renvoie un entier de 0 à 1023, stocké dans la variable valeur.
- **valeur * 5.0 / 1023.0 :** conversion en volts. On utilise des nombres décimaux (5.0 et 1023.0) pour que le résultat soit un nombre à virgule et non un entier tronqué. La variable tension est de type float.
- **lcd.setCursor(colonne, ligne) :** place le curseur d'écriture. (0, 0) correspond à la première ligne, (0, 1) à la deuxième.
- **lcd.print(tension, 2) :** affiche la tension avec 2 décimales. L'espace ajouté après « V » efface les anciens caractères restants, ce qui évite les résidus d'affichage quand le nombre devient plus court.
- **delay(200) :** attend 200 millisecondes avant la mesure suivante.

3.6 Tests et résultats







Observation : la tension affichée suit fidèlement la rotation du potentiomètre, de 0,00 V à 5,00 V, ce qui confirme que la lecture, la conversion et l'affichage fonctionnent correctement.

Lien Wokwi de la simulation 1 : https://wokwi.com/projects/477448790742037505

4. Simulation 2 : la version améliorée

4.1 Améliorations apportées

La seconde version reprend le voltmètre précédent et lui ajoute trois fonctions :

- **Indication visuelle du niveau :** trois LED (verte, jaune, rouge) indiquent si la tension est faible, moyenne ou élevée.
- **Alerte sonore :** un buzzer retentit lorsque la tension atteint le seuil de danger.
- **Bouton HOLD :** un bouton poussoir permet de figer la mesure affichée, comme sur un multimètre réel, puis de reprendre la mesure en direct.

4.2 Composants utilisés

Composant	Qté	Rôle dans le montage
Arduino Uno	1	Lit la tension, applique les seuils et commande toutes les sorties.
Potentiomètre	1	Simule la tension à mesurer (0 à 5 V).
Écran LCD 16x2 (I2C)	1	Affiche la tension, le mode (LIVE ou HOLD) et le niveau.
LED verte	1	S'allume quand la tension est basse (niveau normal).
LED jaune	1	S'allume quand la tension est moyenne.
LED rouge	1	S'allume quand la tension est élevée (danger).
Résistance 220 Ω	3	Limite le courant dans chaque LED pour la protéger.
Buzzer	1	Émet un son d'alerte quand la tension dépasse le seuil de danger.
Bouton poussoir	1	Bouton HOLD : fige ou libère la mesure affichée.

4.3 Schéma de câblage

Le câblage du potentiomètre et de l'écran LCD est identique à celui de la simulation 1. Les éléments ajoutés sont raccordés comme suit :

Composant	Branchement
LED verte	D2 → résistance 220 Ω → LED → GND
LED jaune	D3 → résistance 220 Ω → LED → GND
LED rouge	D4 → résistance 220 Ω → LED → GND
Buzzer	broche + sur D5, broche – sur GND
Bouton HOLD	une patte sur D6, l'autre patte sur GND

4.4 Pourquoi une résistance de 220 Ω pour chaque LED ?

Une LED n'a pratiquement aucune résistance interne : si on la branche directement sur une sortie à 5 V, le courant devient trop grand et peut détruire la LED, voire endommager la broche de l'Arduino. La résistance en série limite ce courant. Avec une LED rouge dont la chute de tension est d'environ 2 V, la loi d'Ohm donne :

$$I = (5\ V - 2\ V) / 220\ \Omega \approx 13{,}6\ mA$$

Cette valeur est suffisante pour que la LED brille correctement, et reste sous la limite de 20 mA recommandée pour une broche de l'Arduino Uno.

4.5 Logique des seuils

Le programme compare la tension mesurée à deux seuils (2 V et 4 V), ce qui définit trois niveaux :

Plage de tension	Niveau	LED allumée	Buzzer	Message sur l'écran
moins de 2 V	Normal	Verte	Éteint	Niveau : NORMAL
de 2 V à moins de 4 V	Moyen	Jaune	Éteint	Niveau : MOYEN
4 V et plus	Danger	Rouge	Actif (1000 Hz)	Niveau : DANGER

Ces seuils ont été choisis pour partager la plage 0 à 5 V en trois zones faciles à observer. Ils peuvent être modifiés facilement en changeant les valeurs 2.0 et 4.0 dans le programme.

4.6 Fonctionnement du bouton HOLD

Le bouton HOLD est relié entre la broche D6 et la masse (GND). La broche est configurée avec INPUT_PULLUP, ce qui active une résistance interne reliant D6 à 5 V. Ainsi :

- bouton relâché : la broche lit HIGH (état haut, grâce à la résistance de rappel interne) ;
- bouton appuyé : la broche est reliée à GND et lit LOW (état bas).

Aucune résistance externe n'est donc nécessaire. Le programme utilise une variable booléenne hold (vrai ou faux) et détecte le front descendant, c'est-à-dire le moment précis où l'état passe de HIGH à LOW. À chaque appui, hold est inversée : le premier appui fige la mesure, le suivant la libère. Un délai de 50 ms (anti-rebond) évite que les petits rebonds mécaniques du bouton soient comptés comme plusieurs appuis.

Quand hold est vrai, la ligne qui met à jour la variable tension n'est plus exécutée : la dernière valeur mesurée est conservée, et l'écran affiche « HOLD » au lieu de « LIVE ». Les LED et le buzzer restent cohérents avec la valeur figée. Pendant le HOLD, tourner le potentiomètre ne change donc rien à l'affichage.

4.7 Programme

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
const int LED_VERTE = 2, LED_JAUNE = 3, LED_ROUGE = 4;
const int BUZZER = 5, BOUTON = 6;

bool hold = false;
bool dernierEtat = HIGH;
float tension = 0;

void setup() {
  pinMode(LED_VERTE, OUTPUT);
  pinMode(LED_JAUNE, OUTPUT);
  pinMode(LED_ROUGE, OUTPUT);
  pinMode(BUZZER, OUTPUT);
  pinMode(BOUTON, INPUT_PULLUP);   // bouton relie a GND

  lcd.init();
  lcd.backlight();
}

void loop() {
  // Gestion du bouton HOLD (appui = bascule figer / reprendre)
  bool etat = digitalRead(BOUTON);
  if (dernierEtat == HIGH && etat == LOW) {
    hold = !hold;
    delay(50);                     // anti-rebond
  }
  dernierEtat = etat;

  // Mesure uniquement si on n'est pas en HOLD
  if (!hold) {
    tension = analogRead(A0) * 5.0 / 1023.0;
  }

  // Affichage
  lcd.setCursor(0, 0);
  lcd.print(hold ? "HOLD  " : "LIVE  ");
  lcd.print(tension, 2);
  lcd.print(" V ");

  lcd.setCursor(0, 1);
  if (tension < 2.0)      lcd.print("Niveau : NORMAL ");
  else if (tension < 4.0) lcd.print("Niveau : MOYEN  ");
  else                    lcd.print("Niveau : DANGER ");

  // LED et buzzer selon la tension
  digitalWrite(LED_VERTE, tension < 2.0);
  digitalWrite(LED_JAUNE, tension >= 2.0 && tension < 4.0);
  digitalWrite(LED_ROUGE, tension >= 4.0);

  if (tension >= 4.0) tone(BUZZER, 1000);
  else noTone(BUZZER);

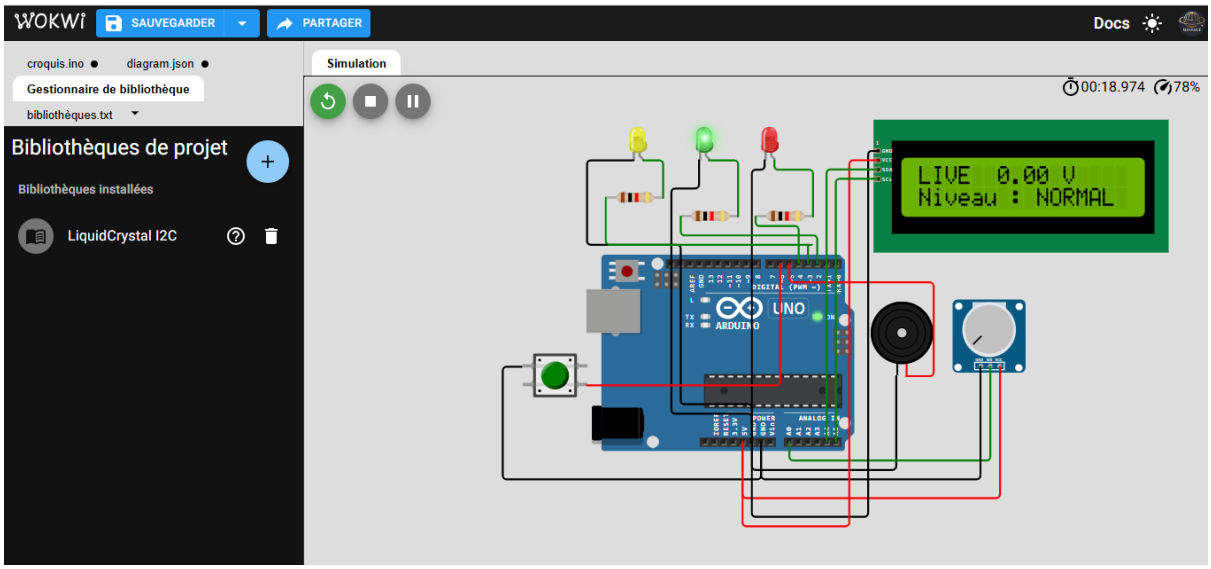
  delay(100);
}
```

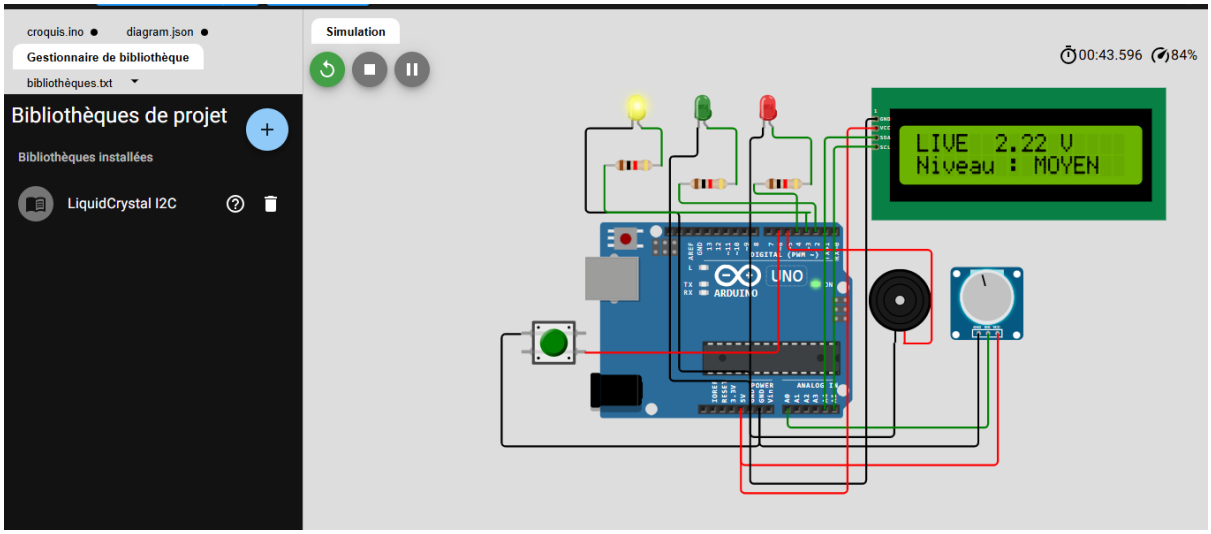
4.8 Explication détaillée du programme

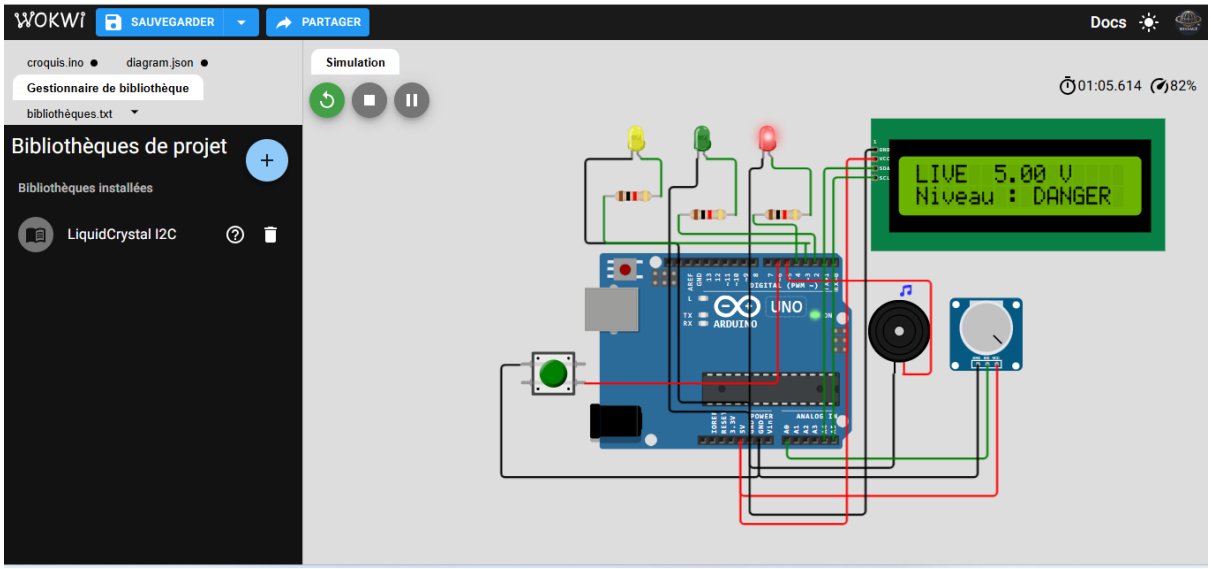
- **Constantes de broches :** LED_VERTE = 2, LED_JAUNE = 3, LED_ROUGE = 4, BUZZER = 5 et BOUTON = 6. Les définir sous forme de constantes rend le programme lisible et facile à modifier.

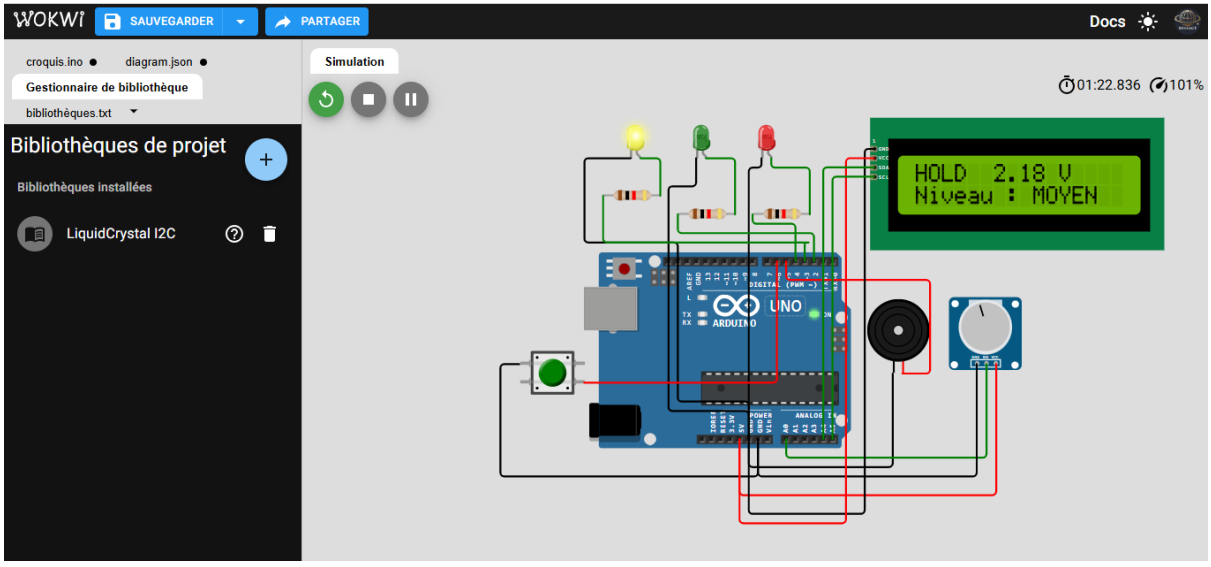
- **Variables globales :** hold mémorise si la mesure est figée ; dernierEtat mémorise l'état précédent du bouton pour détecter un changement ; tension conserve la dernière valeur mesurée.
- **setup() :** configure les LED et le buzzer en sorties (OUTPUT), le bouton en entrée avec résistance de rappel (INPUT_PULLUP), puis initialise l'écran LCD et son rétroéclairage.
- **Lecture du bouton :** digitalRead(BOUTON) lit l'état actuel. La condition (dernierEtat == HIGH && etat == LOW) n'est vraie qu'à l'instant de l'appui. On inverse alors hold avec hold = !hold, puis on attend 50 ms pour l'anti-rebond. Enfin dernierEtat est mis à jour.
- **Mesure conditionnelle :** if (!hold) ne met à jour la tension que si l'on n'est pas en mode HOLD.
- **Affichage ligne 1 :** l'opérateur ternaire hold ? "HOLD " : "LIVE " choisit le mot à afficher selon le mode, suivi de la tension avec deux décimales.
- **Affichage ligne 2 :** une suite de if / else if / else sélectionne le message NORMAL, MOYEN ou DANGER selon les seuils. Les textes ont tous la même longueur (16 caractères) pour écraser complètement l'ancien message.
- **Commande des LED :** digitalWrite(pin, condition) reçoit un résultat booléen : vrai allume la LED (HIGH), faux l'éteint (LOW). Une seule condition est vraie à la fois, donc une seule LED est allumée.
- **Buzzer :** tone(BUZZER, 1000) génère un son de 1000 Hz tant que la tension est supérieure ou égale à 4 V ; noTone(BUZZER) coupe le son dans les autres cas.
- **delay(100) :** petite pause de 100 ms entre deux tours de boucle.

4.9 Tests et résultats









Lien Wokwi de la simulation 2 : https://wokwi.com/projects/477453117099021313

5. Synthèse : rôle de chaque composant

Composant	Rôle	Que se passerait-il sans lui ?
Arduino Uno	Lit, calcule, décide et commande toutes les sorties.	Aucun traitement possible : le montage ne fonctionne pas.
Potentiomètre	Fournit une tension variable de 0 à 5 V (diviseur de tension).	Aucune tension à mesurer.
Entrée A0 (CAN)	Convertit la tension en nombre de 0 à 1023.	L'Arduino ne pourrait pas lire la tension.
Écran LCD I2C	Affiche la tension et les informations de mode.	Le résultat serait invisible pour l'utilisateur.
LED verte, jaune, rouge	Indiquent le niveau de tension d'un coup d'œil.	Plus d'indication visuelle rapide du niveau.
Résistances 220 Ω	Limitent le courant dans les LED.	Risque de surintensité et de destruction des LED.
Buzzer	Alerte sonore en cas de tension élevée.	Pas d'alerte si l'on ne regarde pas le montage.
Bouton HOLD	Fige et libère la mesure affichée.	Impossible de lire calmement une valeur qui varie.

7. Conclusion

Ce devoir a permis de réaliser en simulation un voltmètre numérique complet à partir d'une carte Arduino Uno. La première simulation a montré le principe de base : la tension d'un potentiomètre est lue par le convertisseur analogique-numérique, convertie en volts par calcul, puis affichée sur un écran LCD relié en I2C.

La seconde simulation a enrichi ce montage : les trois LED et le buzzer traduisent la mesure en information visuelle et sonore à l'aide de seuils, et le bouton HOLD, géré avec une résistance de rappel interne et un anti-rebond, permet de figer la valeur affichée. Chaque composant a un rôle précis, et le programme coordonne l'ensemble de manière cohérente.

Ce travail illustre les bases de l'instrumentation : mesurer, traiter, afficher et signaler. Pour aller plus loin, on pourrait ajouter un pont diviseur pour mesurer des tensions supérieures à 5 V, ou utiliser un module de mesure dédié pour augmenter la précision.